



## Day 2 | AI Agent 協作與 Web 架構

先讀懂、再決策；先設計、再授權 AI 實作

# 昨天，我們已完成第一個公開版本

- 用清楚需求指揮 AI 產生網站
- 以驗收條件檢查畫面與互動

# 今天，你要學會做出「有理由的下一步」

- 理解前端、後端、資料庫與 API 的責任
- **請 AI 生成文件，但先不寫程式**
- 理解需求、建立文件與記憶、規劃開發流程

# AI Agent 工具

# 各類 AI 工具付費比較

| 工具                                 | 訂閱    | 教學影片                                                                                                  |
|------------------------------------|-------|-------------------------------------------------------------------------------------------------------|
| <a href="#">ChatGPT 桌面版</a>        | 免訂閱可用 | <a href="https://www.youtube.com/watch?v=SFWYaF0HJiw">https://www.youtube.com/watch?v=SFWYaF0HJiw</a> |
| <a href="#">Google Antigravity</a> | 免訂閱可用 | <a href="https://www.youtube.com/watch?v=v0Y9zMJuYb0">https://www.youtube.com/watch?v=v0Y9zMJuYb0</a> |
| <a href="#">Claude cowork</a>      | 需訂閱   | <a href="https://www.youtube.com/watch?v=_cVTzXvb7xs">https://www.youtube.com/watch?v=_cVTzXvb7xs</a> |
| GitHub Copilot (vscode)            | 免訂閱可用 | <a href="https://www.youtube.com/watch?v=mFCryoMibBo">https://www.youtube.com/watch?v=mFCryoMibBo</a> |

# Web application 基本概念

# Part 1 | Web 應用程式是什麼？

## 你每天都在用 Web 應用程式

Instagram、YouTube、Google Maps

學校的選課系統、圖書館查詢系統

任何你用「瀏覽器」開啟的服務

**共同點：** 你的電腦只是「畫面」，真正的資料和邏輯在別的地方。

# Web 應用程式的三層架構





# 前端 Frontend

負責：讓使用者「看到」與「操作」

| 技術         | 作用                     |
|------------|------------------------|
| HTML       | 定義頁面結構(標題、按鈕、輸入框)      |
| CSS        | 控制外觀(顏色、排版、動畫)         |
| JavaScript | 互動行為(點擊、呼叫後端 API、更新畫面) |

本課程的前端：純 HTML + JavaScript(不使用 React、Vue 等前端框架)

# 後端 Backend

負責：處理邏輯、回應請求

**本課程使用：Python + FastAPI**

- FastAPI 是一個現代、快速(高效能)的 Web 框架, 用於以 Python 並基於標準的 Python 型別提示來構建 API。



# 資料庫 Database

**負責:永久儲存資料**

**本地使用:SQLite**

一個檔案(.db)就是一個完整資料庫

不需要額外安裝伺服器

適合小型專案、學習用途

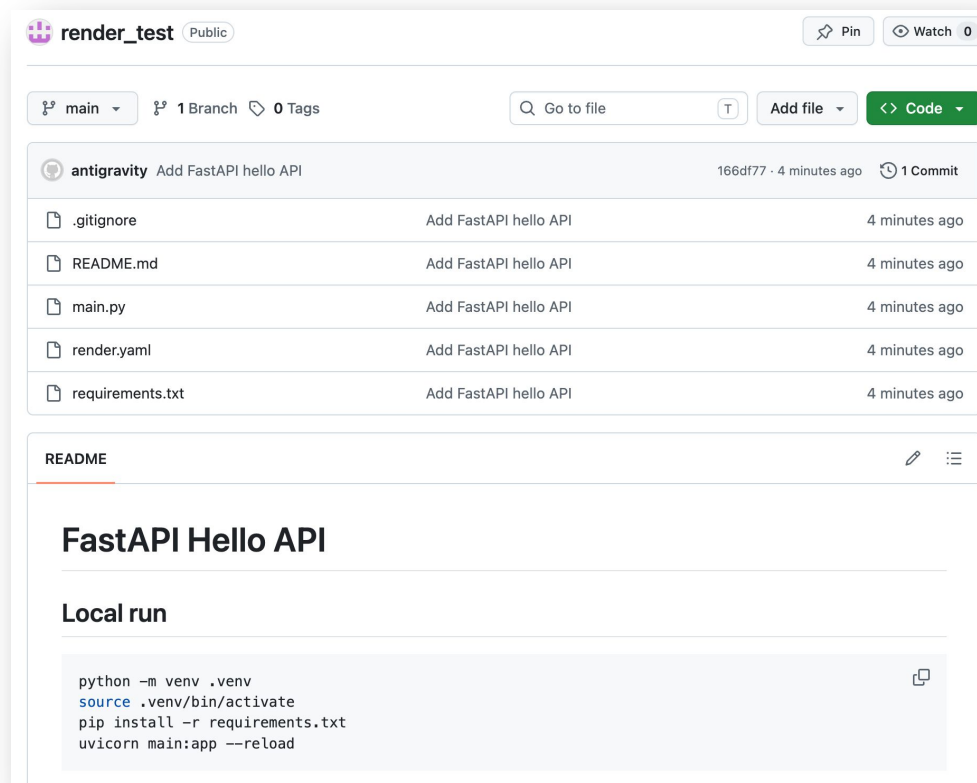
**部署使用:render PostgreSQL**

**免費方案資料只能儲存 30天**

# 簡易後端

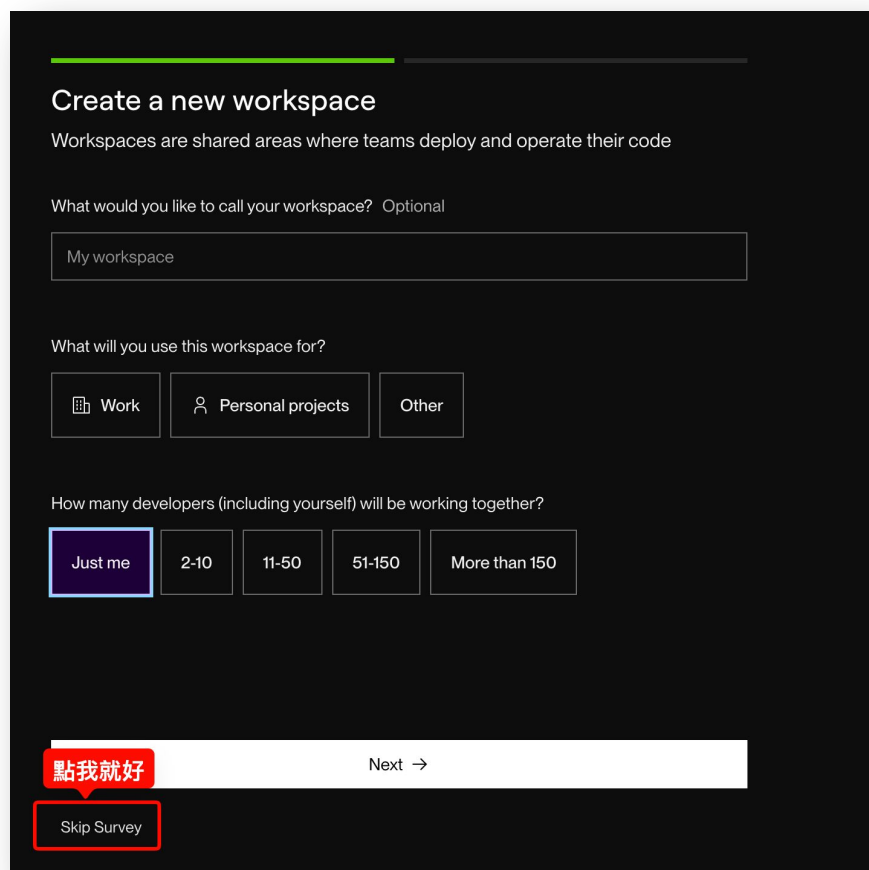
# Try it: 先開發一個簡單的後端程式，並部署到 render

- 先建立一個「render\_test」專案
- 讓 AI 使用「fastapi」實作一個要部署到「render」的回傳「Hello」的 api
- 推送到 GitHub



# Try it: 先開發一個簡單的後端程式，並部署到 render

- 前往：<https://render.com/>
- 註冊並登入(可直接使用github帳號登入)



The screenshot shows the 'Create a new workspace' form on the Render website. The form is dark-themed with white text. It includes a title 'Create a new workspace', a subtitle 'Workspaces are shared areas where teams deploy and operate their code', and a question 'What would you like to call your workspace? Optional' with a text input field containing 'My workspace'. Below this is another question 'What will you use this workspace for?' with three buttons: 'Work', 'Personal projects', and 'Other'. The 'Work' button is selected. The next question is 'How many developers (including yourself) will be working together?' with five buttons: 'Just me', '2-10', '11-50', '51-150', and 'More than 150'. The 'Just me' button is selected. At the bottom, there is a 'Next →' button and a 'Skip Survey' button. A red box highlights the 'Skip Survey' button, and a red speech bubble with the text '點我就好' (Click me) points to it.

Create a new workspace

Workspaces are shared areas where teams deploy and operate their code

What would you like to call your workspace? Optional

My workspace

What will you use this workspace for?

Work Personal projects Other

How many developers (including yourself) will be working together?

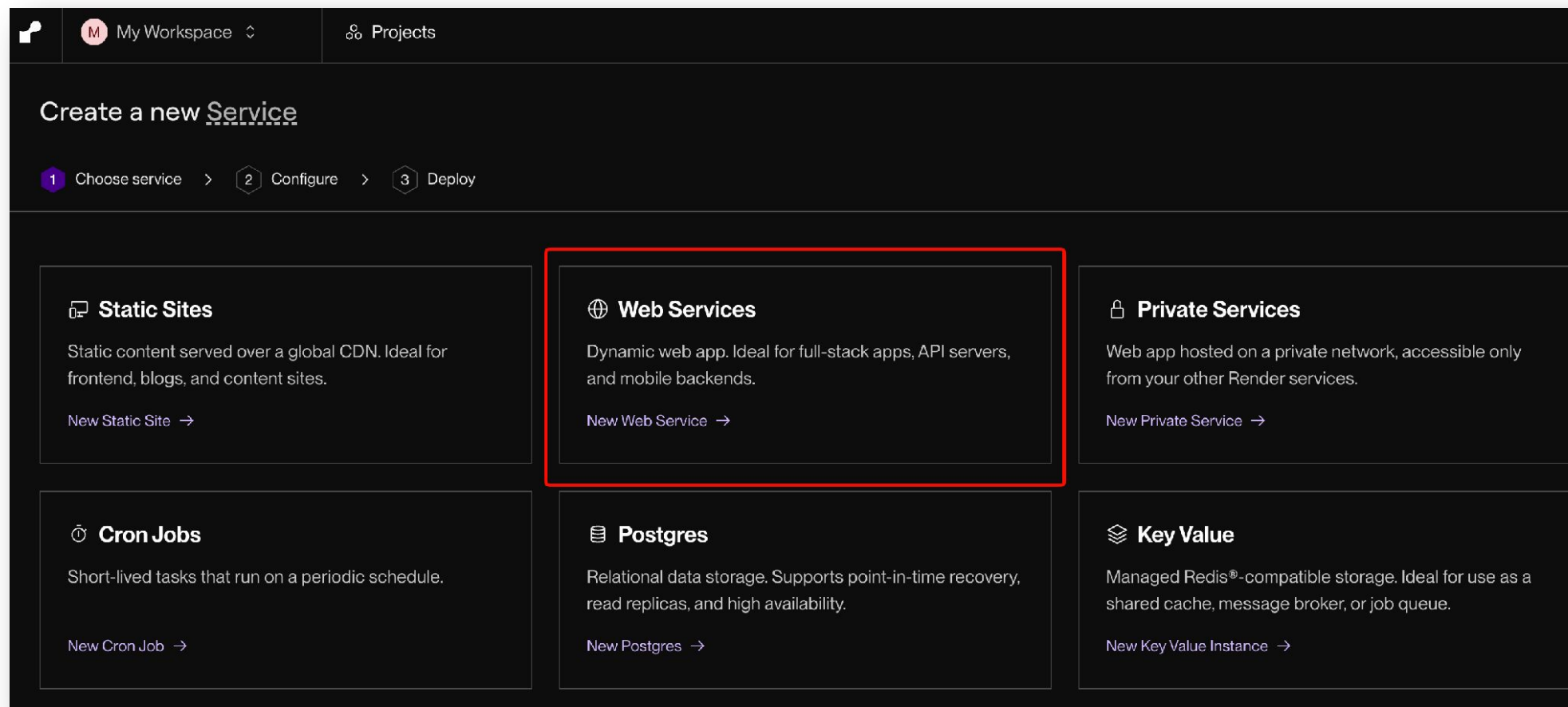
Just me 2-10 11-50 51-150 More than 150

Next →

Skip Survey

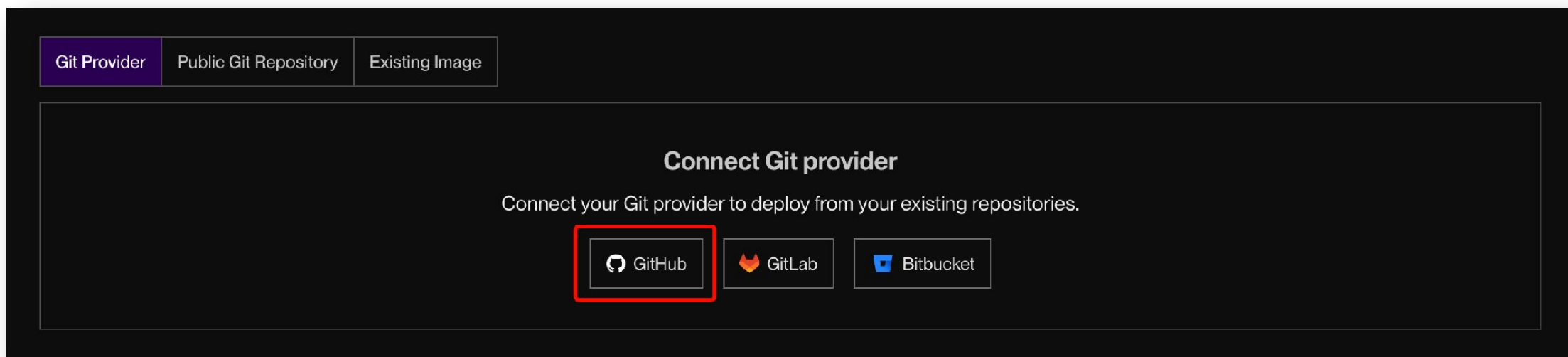
# Try it: 先開發一個簡單的後端程式，並部署到 render

- 建立「Web Services」

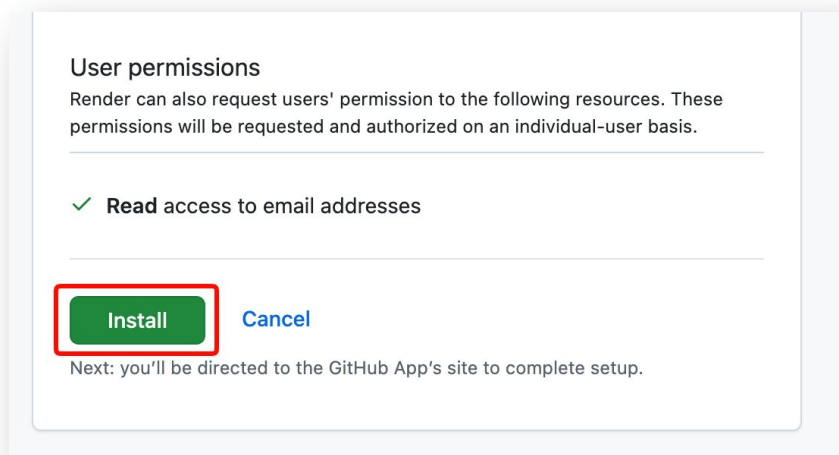


# Try it: 先開發一個簡單的後端程式，並部署到 render

- 選擇「GitHub」



- 下滑並點擊「Install」





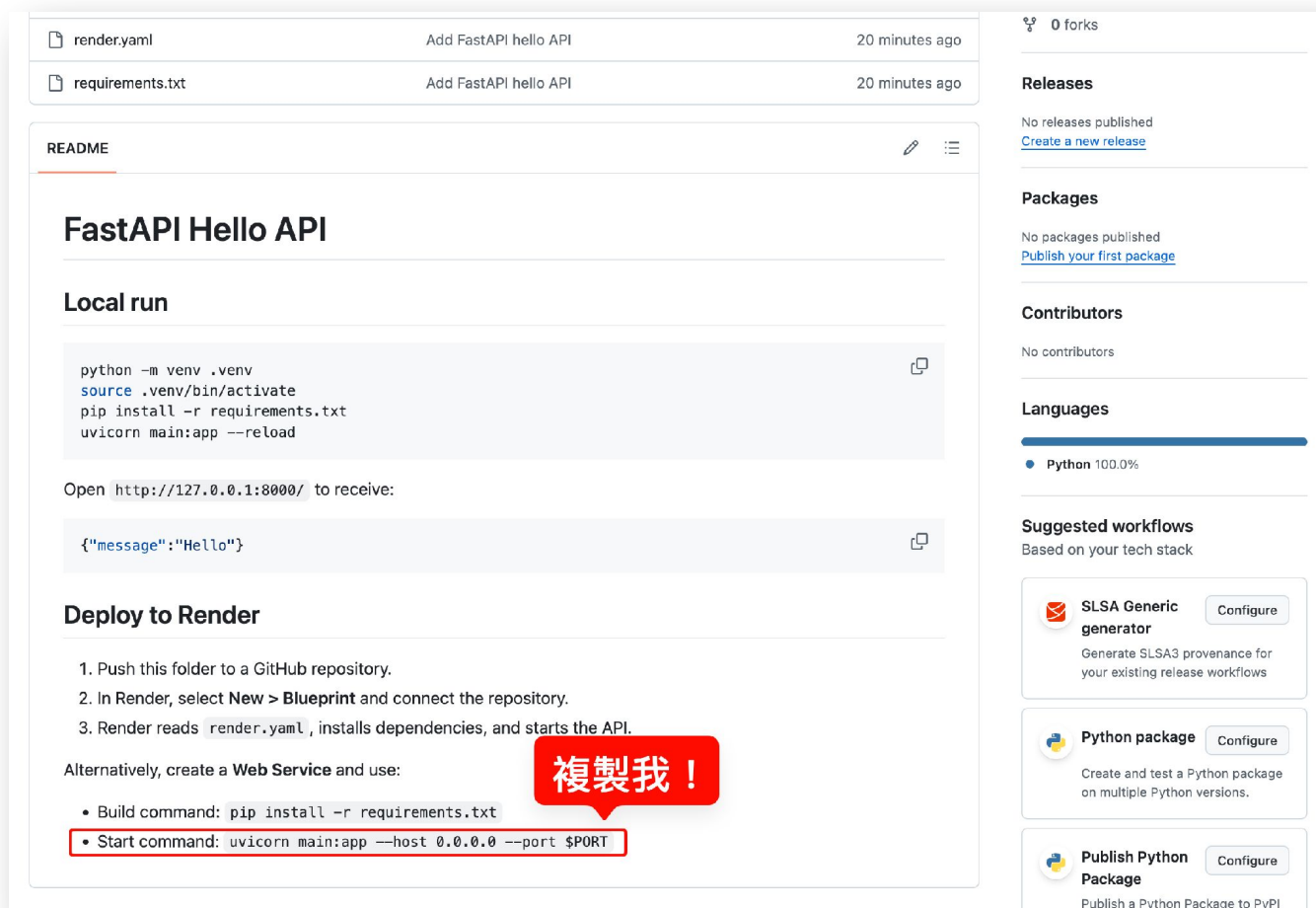
# Try it: 先開發一個簡單的後端程式，並部署到 render

- 選擇你的專案



# Try it: 先開發一個簡單的後端程式，並部署到 render

- 看 GitHub 專案的「README」，並找到紅框中以「unicorn」開頭的執行指令  
並複製



render.yaml Add FastAPI hello API 20 minutes ago

requirements.txt Add FastAPI hello API 20 minutes ago

## FastAPI Hello API

### Local run

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
```

Open `http://127.0.0.1:8000/` to receive:

```
{"message": "Hello"}
```

### Deploy to Render

1. Push this folder to a GitHub repository.
2. In Render, select **New > Blueprint** and connect the repository.
3. Render reads `render.yaml`, installs dependencies, and starts the API.

Alternatively, create a **Web Service** and use:

- Build command: `pip install -r requirements.txt`
- Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`

0 forks

### Releases

No releases published  
[Create a new release](#)

### Packages

No packages published  
[Publish your first package](#)

### Contributors

No contributors

### Languages

Python 100.0%

### Suggested workflows

Based on your tech stack

- SLSA Generic generator** [Configure](#)  
Generate SLSA3 provenance for your existing release workflows
- Python package** [Configure](#)  
Create and test a Python package on multiple Python versions.
- Publish Python Package** [Configure](#)  
Publish a Python Package to PyPI

# Try it: 先開發一個簡單的後端程式，並部署到 render

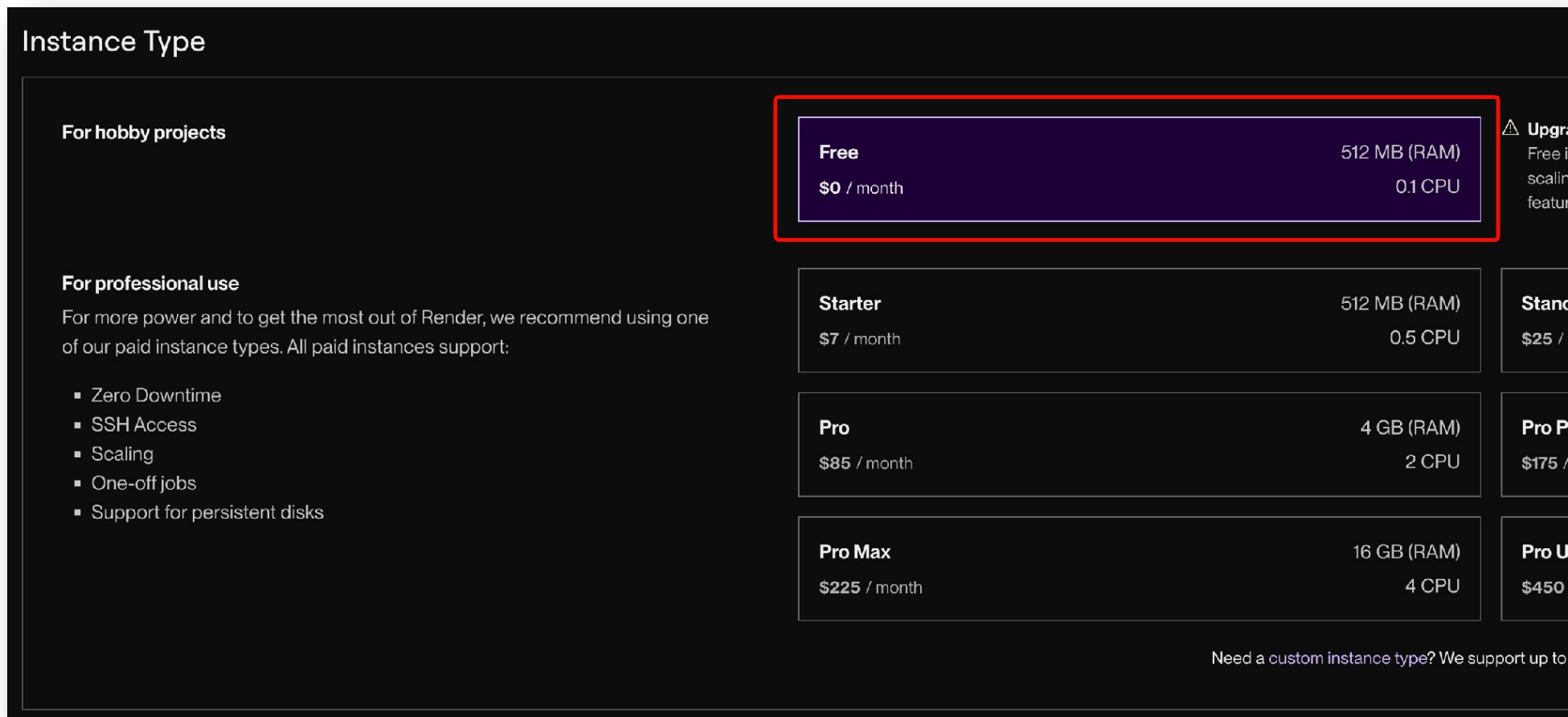
- 找到「Start Command」，並將指令貼上到後方輸入框中

|                                                                                                                                                                                                                                            |                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Branch</b><br>The Git branch to build and deploy.                                                                                                                                                                                       | <input type="text" value="main"/>                               |
| <b>Region</b><br>Your services in the same region can communicate over a private network.                                                                                                                                                  | <input type="text" value="Oregon (US West)"/>                   |
| <b>Root Directory</b> Optional<br>If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo. | <input type="text" value="e.g. src"/>                           |
| <b>Build Command</b><br>Render runs this command to build your app before each deploy.                                                                                                                                                     | <input type="text" value="\$ pip install -r requirements.txt"/> |
| <b>Start Command</b><br>Render runs this command to start your app with each deploy.                                                                                                                                                       | <input type="text" value="\$ uvicorn main:app --reload"/>       |

貼上到這裡

# Try it: 先開發一個簡單的後端程式，並部署到 render

- 選擇「Free」方案



**Instance Type**

**For hobby projects**

| Plan           | Price         | RAM          | CPU     |
|----------------|---------------|--------------|---------|
| <b>Free</b>    | \$0 / month   | 512 MB (RAM) | 0.1 CPU |
| <b>Starter</b> | \$7 / month   | 512 MB (RAM) | 0.5 CPU |
| <b>Pro</b>     | \$85 / month  | 4 GB (RAM)   | 2 CPU   |
| <b>Pro Max</b> | \$225 / month | 16 GB (RAM)  | 4 CPU   |

**For professional use**

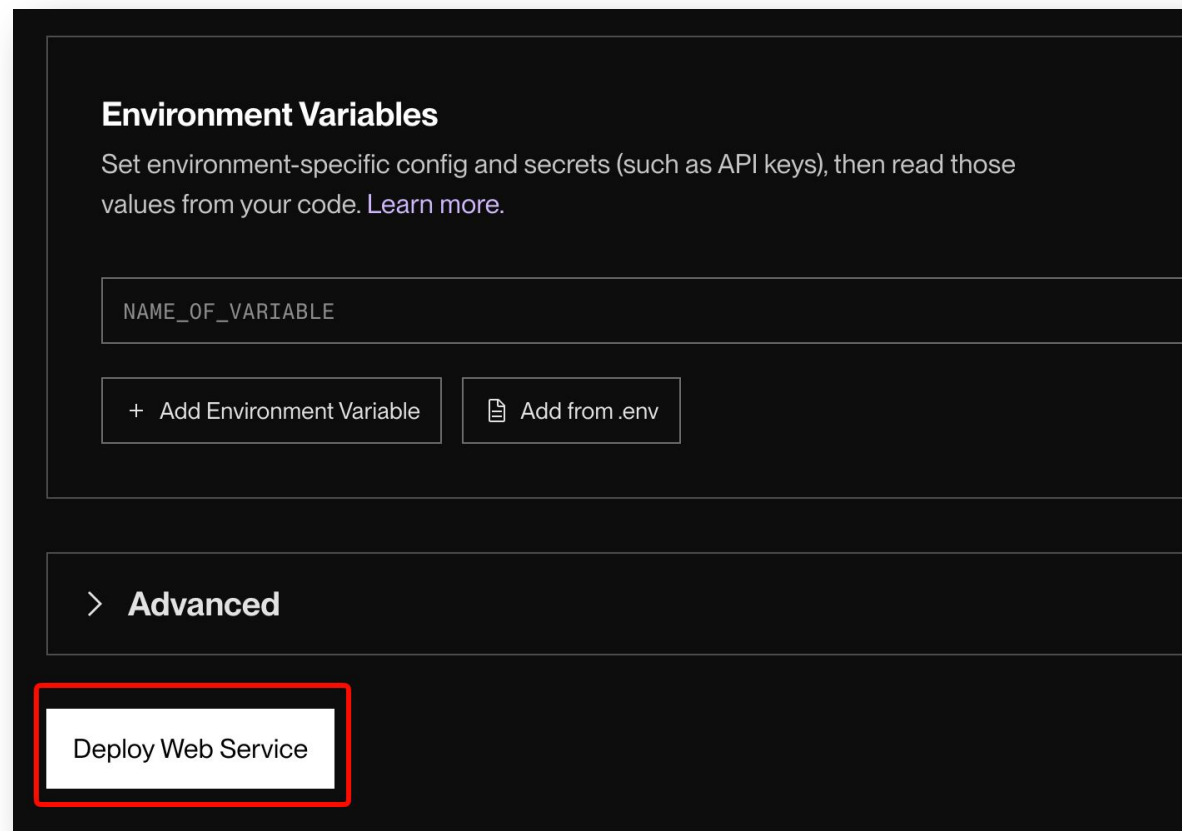
For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Need a custom instance type? We support up to

# Try it: 先開發一個簡單的後端程式，並部署到 render

- 按下「Deploy Web Service」



## Try it: 先開發一個簡單的後端程式，並部署到 render

- 觀察下方 log 區塊，如果顯示「Available at your primary URL ...」表示成功

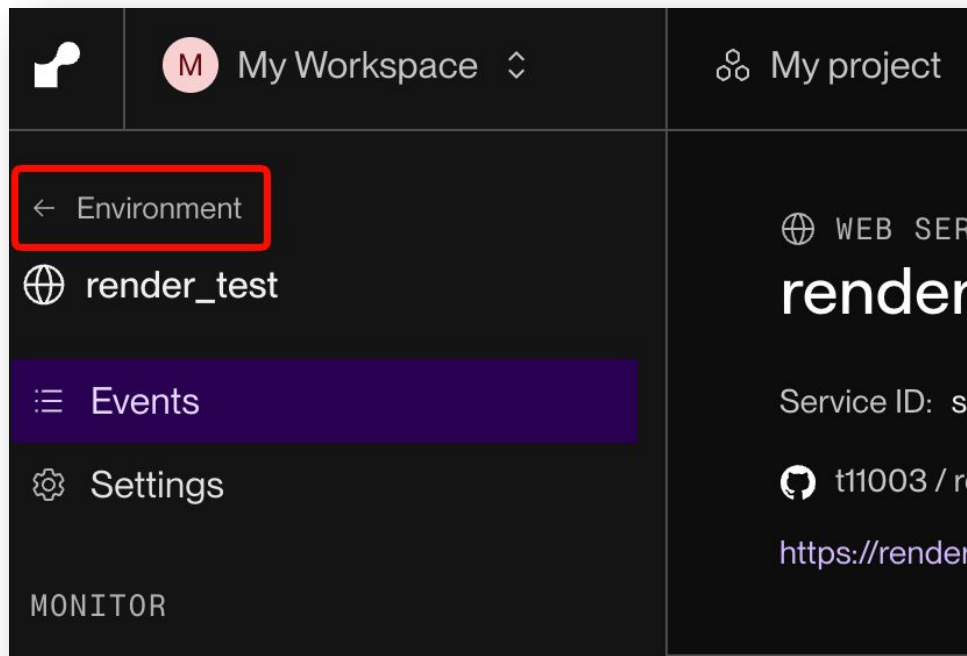
```
==> Your service is live 🎉  
==>  
==> ///////////////////////////////////  
==>  
==> Available at your primary URL https://render-test-j2u3.onrender.com  
==>  
==> //////////////////////////////////
```

- 點擊網址會看到「Hello」，表示部署成功並正確執行。

# render Postgresql

# Try it: 啟用並連接 render PostgreSQL 資料庫

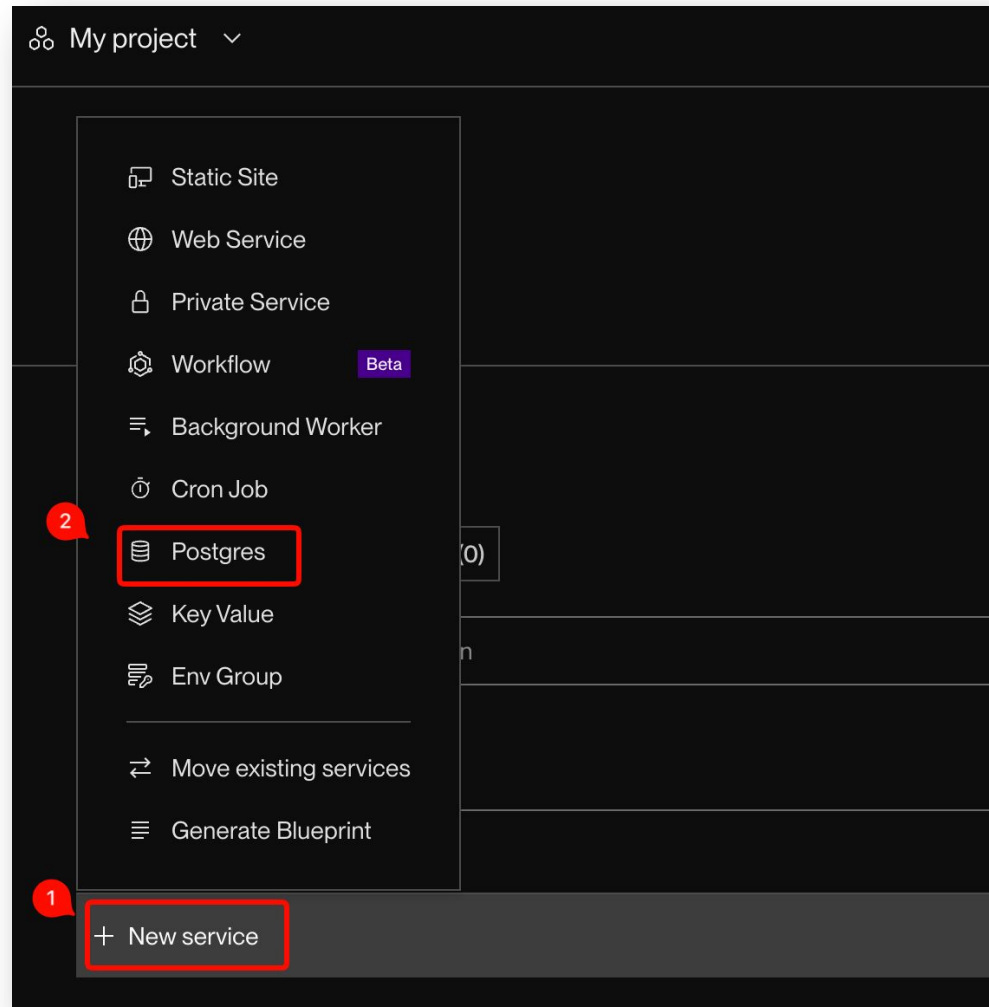
- 點擊「render.com」左上方的「← Enviroment」回到 project





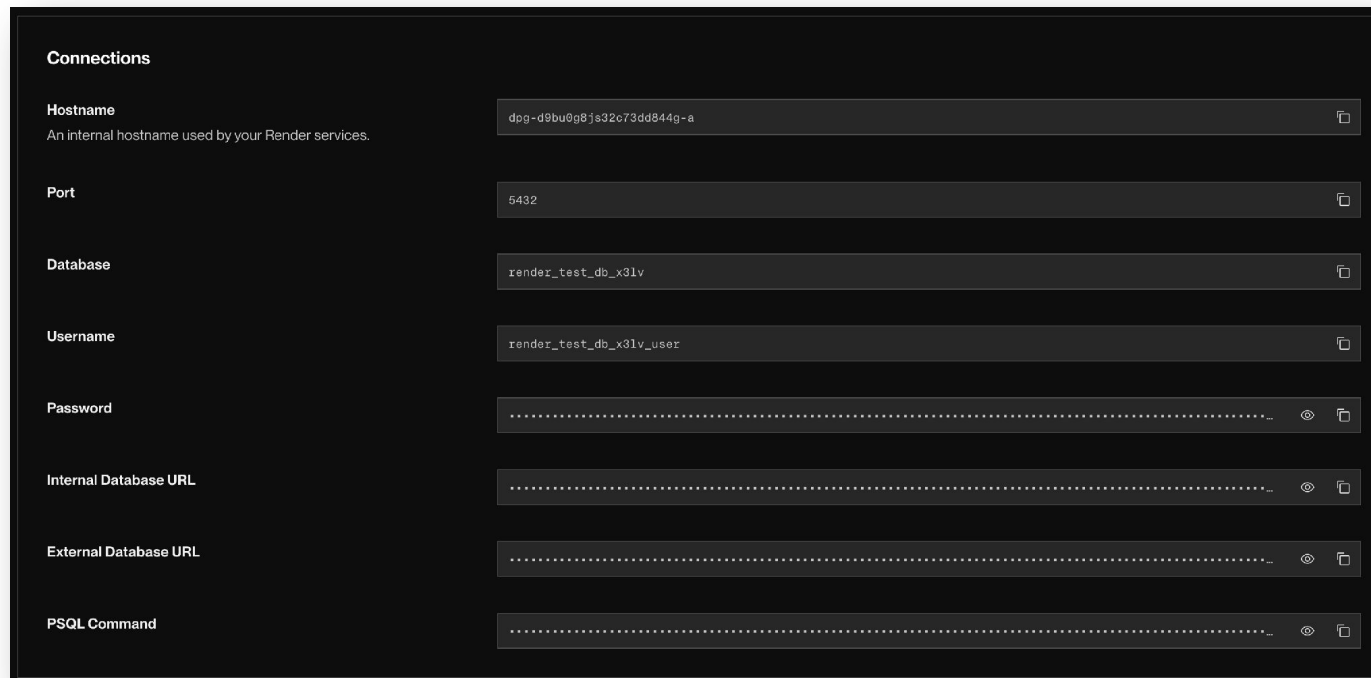
# Try it: 啟用並連接 render Postgresql 資料庫

- 點「+New service」=>「Postgres」



# Try it: 啟用並連接 render PostgreSQL 資料庫

- 「Name」後方輸入框輸入資料庫名稱, 例如「render\_test\_db」
- 「Instance Type」選擇「Free」方案
- 按下底部「Create Database」按鈕
- 等待資料庫啟動完成後會是下列畫面



The screenshot displays the 'Connections' section of the Render PostgreSQL interface. It lists the following connection details:

| Field                 | Value                      |
|-----------------------|----------------------------|
| Hostname              | dpg-d9bu8g8js32c73dd844g-a |
| Port                  | 5432                       |
| Database              | render_test_db_x3lv        |
| Username              | render_test_db_x3lv_user   |
| Password              | .....                      |
| Internal Database URL | .....                      |
| External Database URL | .....                      |
| PSQL Command          | .....                      |

# Try it: 啟用並連接 render Postgresql 資料庫

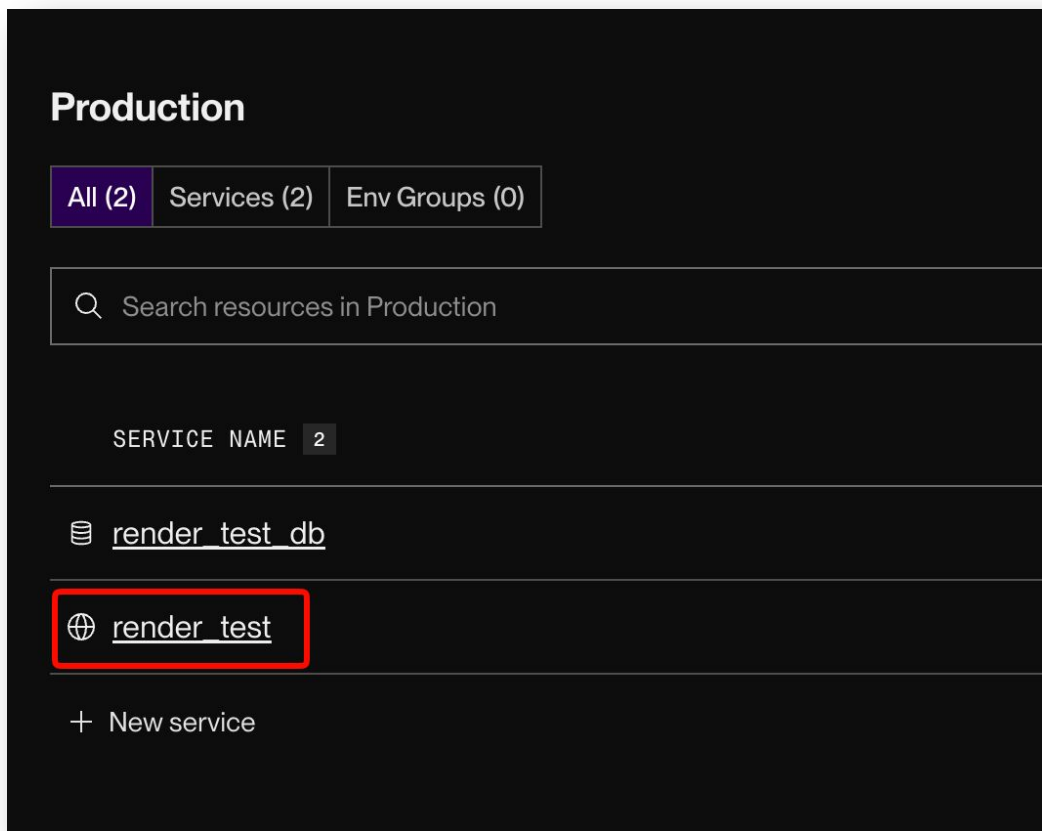
- 複製「Internal Database URL」

**Connections**

|                                                                       |                            |      |
|-----------------------------------------------------------------------|----------------------------|------|
| <b>Hostname</b><br>An internal hostname used by your Render services. | dpg-d9bu0g8js32c73dd844g-a | 📋    |
| <b>Port</b>                                                           | 5432                       | 📋    |
| <b>Database</b>                                                       | render_test_db_x3lv        | 📋    |
| <b>Username</b>                                                       | render_test_db_x3lv_user   | 📋    |
| <b>Password</b>                                                       | .....                      | 👁️ 📋 |
| <b>Internal Database URL</b>                                          | .....                      | 👁️ 📋 |
| <b>External Database URL</b>                                          | .....                      | 👁️ 📋 |
| <b>PSQL Command</b>                                                   | .....                      | 👁️ 📋 |

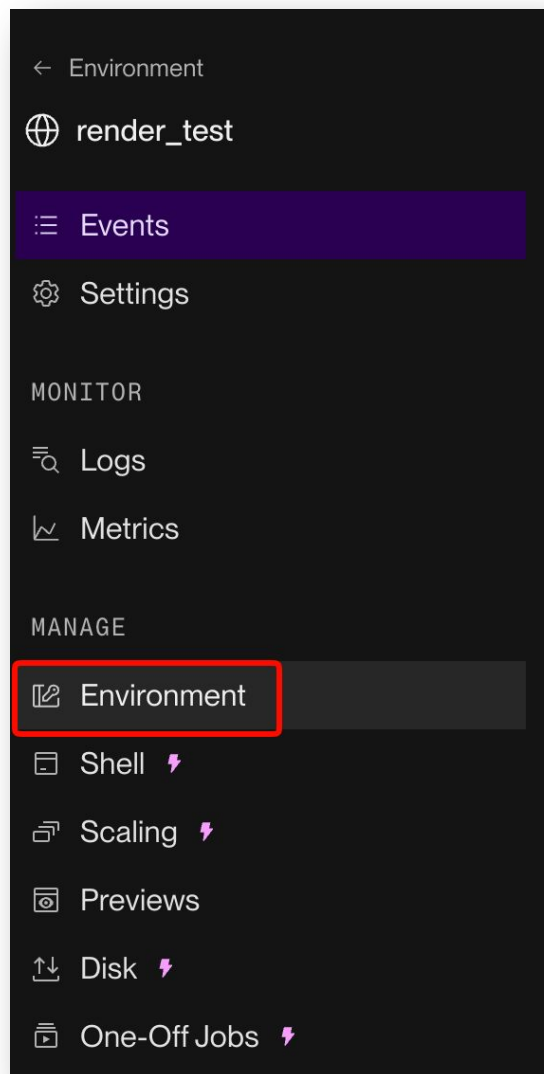
# Try it: 啟用並連接 render PostgreSQL 資料庫

- 點擊左上方的「← Enviroment」回到 project
- 選擇你的「Web service」



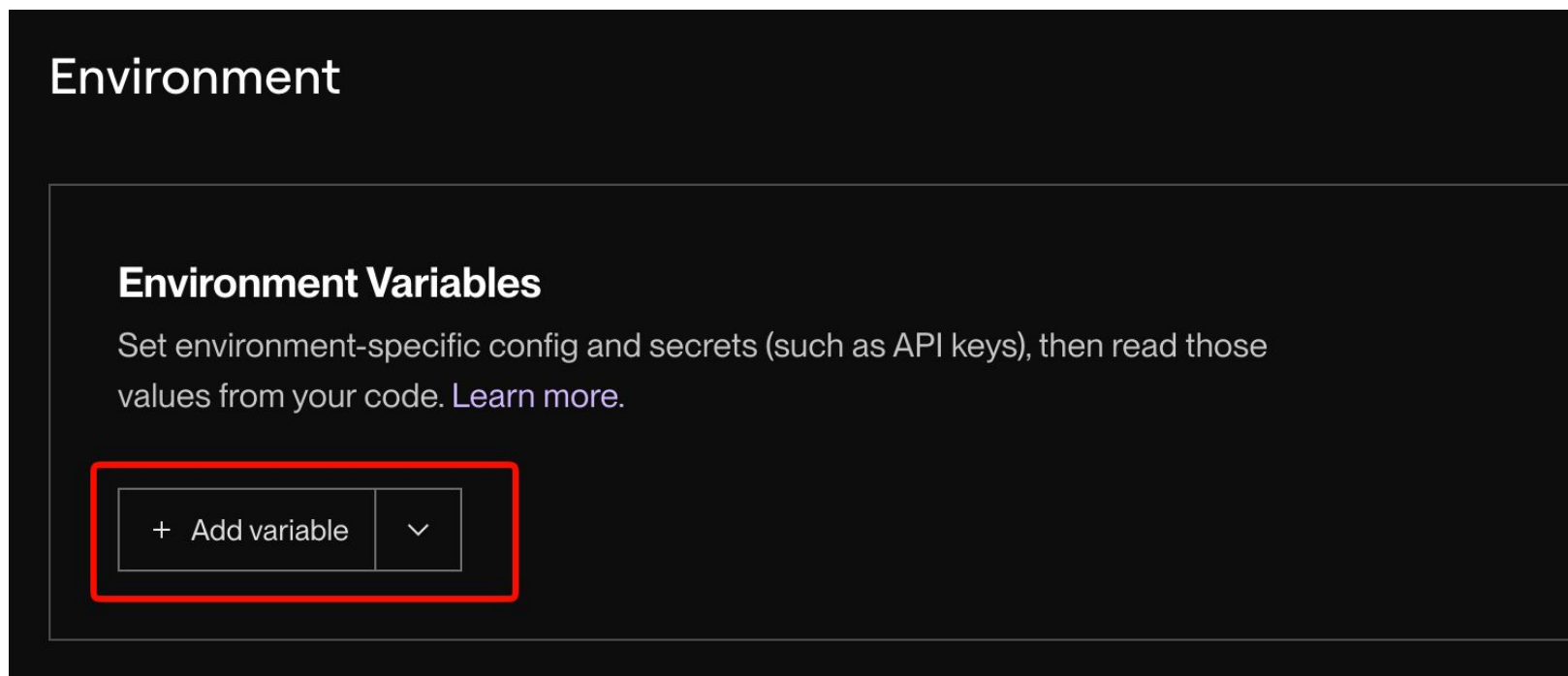
# Try it: 啟用並連接 render Postgresql 資料庫

- 找到左邊的「Enviroment」



# Try it: 啟用並連接 render Postgresql 資料庫

- 找到「Environment Variables」, 並點「Add variable」



# Try it: 啟用並連接 render Postgresql 資料庫

- KEY 輸入: DATABASE\_URL
- VALUE 輸入: 剛剛複製的 Internal Database URL

**Environment Variables**

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

| KEY                                                   | VALUE                                                       |
|-------------------------------------------------------|-------------------------------------------------------------|
| <div>1 輸入「DATABASE_URL」</div> <div>DATABASE_URL</div> | <div>2 貼上剛剛複製的 Internal Database URL</div> <div>.....</div> |

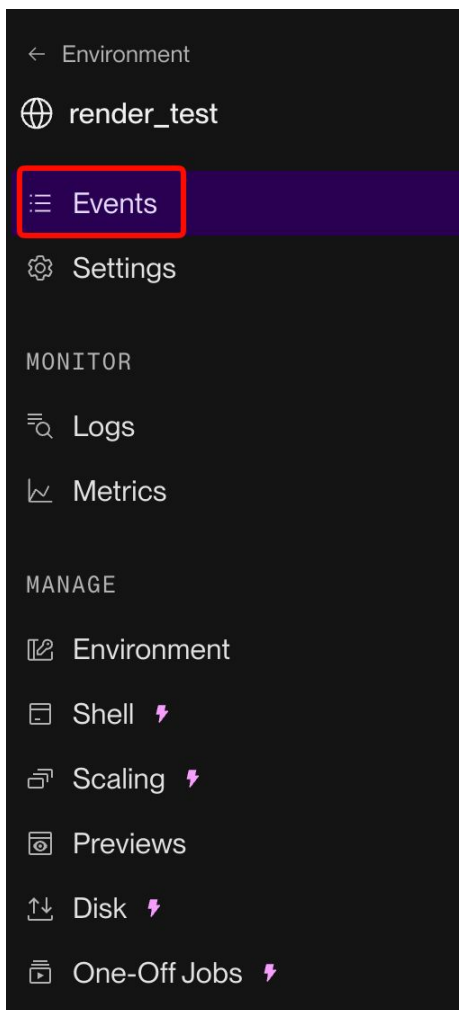
+ Add variable ▼

3 點擊此按鈕

Save, rebuild, and deploy ▼

# Try it: 啟用並連接 render Postgresql 資料庫

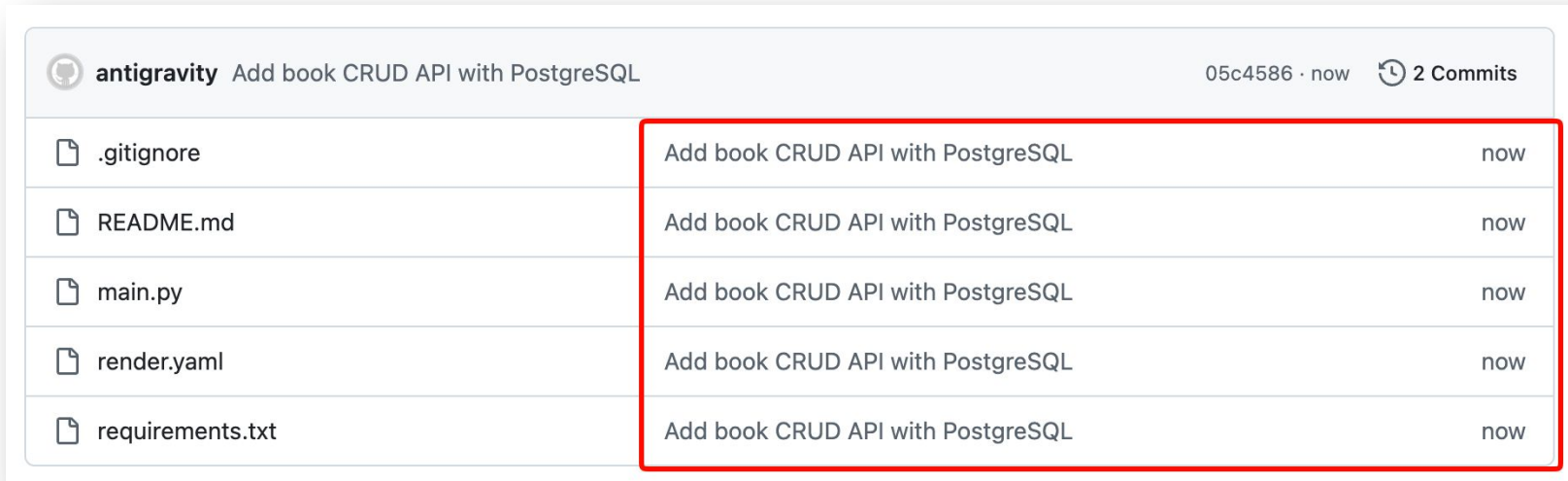
- 找到左邊的「Event」，等待最新的「Deploy」完成





# Try it: 啟用並連接 render Postgresql 資料庫

- 要求 AI 建立一個「book」表格的 crud api, 使用 render Postgresql。
- 推送到 GitHub。
- 如果出現下圖代表推送成功



The screenshot shows a GitHub commit page for the repository 'antigravity' with the commit message 'Add book CRUD API with PostgreSQL'. The commit hash is '05c4586' and it was made 'now'. There are '2 Commits' in total. The commit details table is highlighted with a red border.

| File             | Commit Message                    | Time |
|------------------|-----------------------------------|------|
| .gitignore       | Add book CRUD API with PostgreSQL | now  |
| README.md        | Add book CRUD API with PostgreSQL | now  |
| main.py          | Add book CRUD API with PostgreSQL | now  |
| render.yaml      | Add book CRUD API with PostgreSQL | now  |
| requirements.txt | Add book CRUD API with PostgreSQL | now  |

# Try it: 如果編譯失敗

- 點擊「deploy logs」查看原因

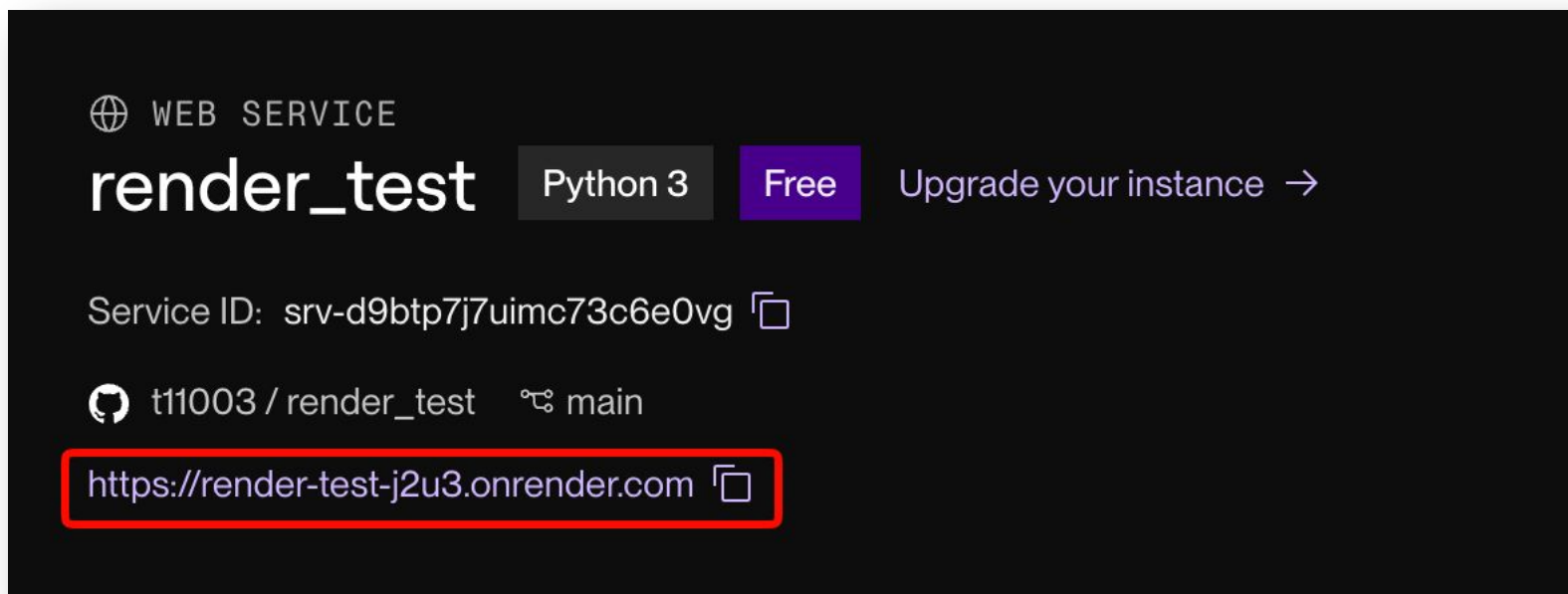
Deploy failed for 05c4586: Add book CRUD API with PostgreSQL  
✖ Exited with status 1 while building your code. Check your deploy logs for more information.

- 複製錯誤訊息給 AI，不要只複製出錯的，需連同上下沒有紅框的一起。

```
03:45:48 AM Using cached pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
03:45:48 AM Collecting typing-extensions>=4.8.0 (from fastapi==0.116.1->-r requirements.txt (line 1))
03:45:48 AM Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
03:45:48 AM Collecting click>=7.0 (from uvicorn==0.35.0->-r requirements.txt (line 4))
03:45:48 AM Using cached click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
03:45:48 AM Collecting h11>=0.8 (from uvicorn==0.35.0->uvicorn[standard]==0.35.0->-r requirements.txt (line 4))
03:45:48 AM Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
03:45:48 AM INFO: pip is looking at multiple versions of psycogp[binary] to determine which version is compatible with other requirements. This c
✖ 03:45:48 AM ERROR: Could not find a version that satisfies the requirement psycogp-binary==3.2.9; implementation_name != "pypy" and extra == "bin
3.2.11, 3.2.12, 3.2.13, 3.3.0, 3.3.1, 3.3.2, 3.3.3, 3.3.4)
03:45:48 AM
03:45:48 AM [notice] A new release of pip is available: 25.3 -> 26.1.2
03:45:48 AM [notice] To update, run: pip install --upgrade pip
✖ 03:45:48 AM ERROR: No matching distribution found for psycogp-binary==3.2.9; implementation_name != "pypy" and extra == "binary"
03:45:48 AM ==> Build failed 🙄
03:45:48 AM ==> Common ways to troubleshoot your deploy: https://render.com/docs/troubleshooting-deploys
```

## Try it: 編譯成功後

- 點擊網址查看頁面

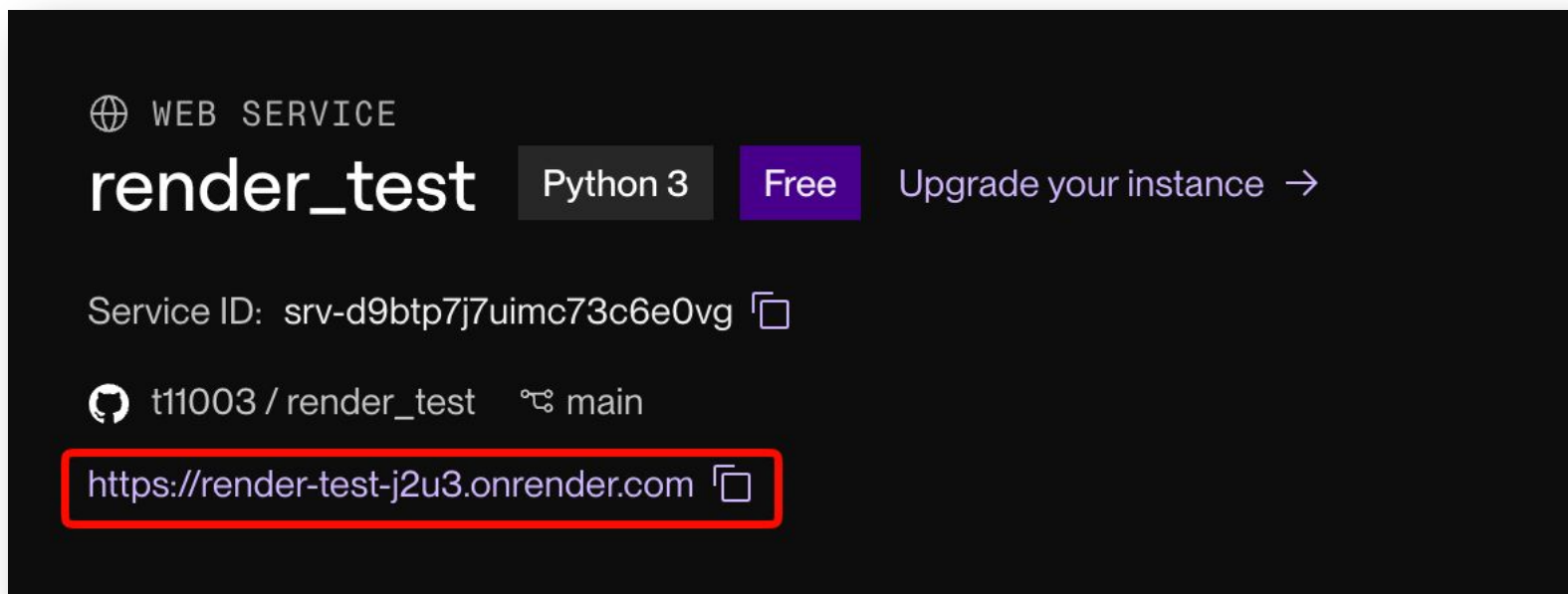


- 在瀏覽器的網址列後面加上「/docs」



## Try it: 編譯成功後

- 點擊網址查看頁面



- 在瀏覽器的網址列後面加上「/docs」



# Try it: 編譯成功後

- 會看到所有你後端服務 API 的端點, 也可以進行測試

## Book CRUD API

0.1.0 OAS 3.1

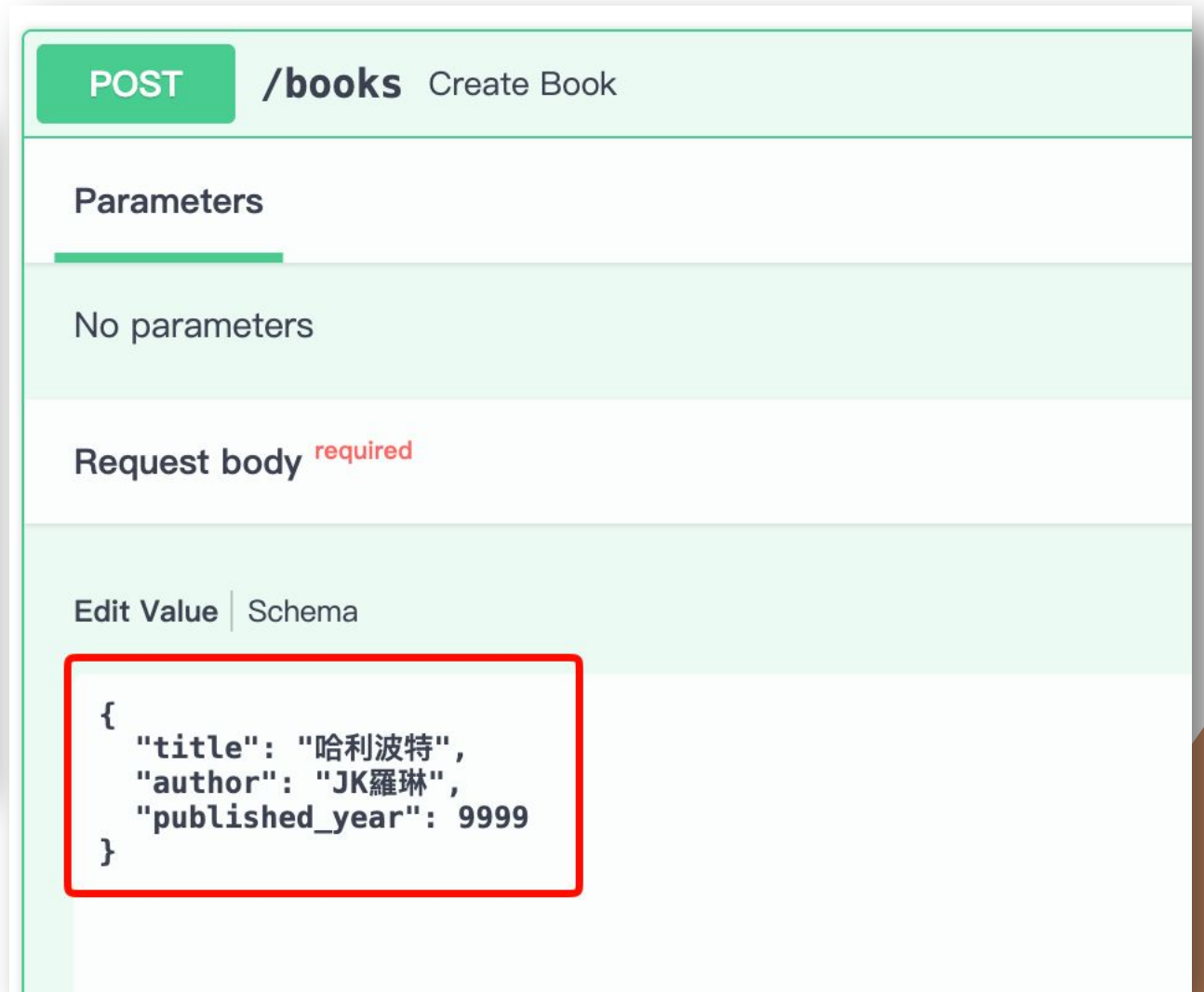
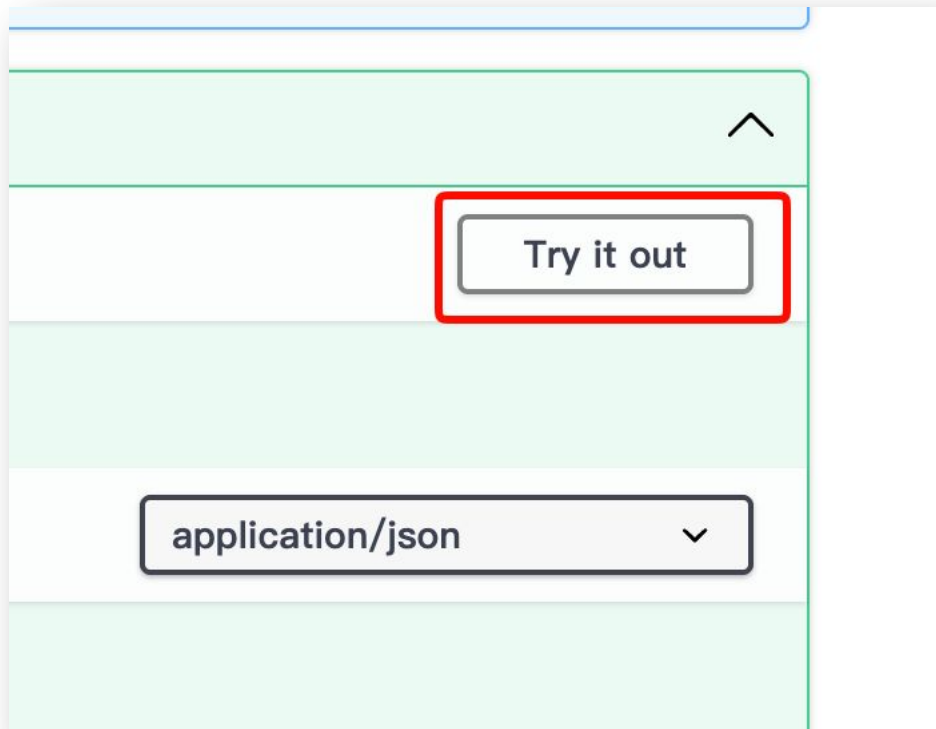
[/openapi.json](#)

default ^

|        |                              |   |
|--------|------------------------------|---|
| GET    | / Hello                      | ✓ |
| GET    | /books List Books            | ✓ |
| POST   | /books Create Book           | ✓ |
| GET    | /books/{book_id} Get Book    | ✓ |
| PUT    | /books/{book_id} Update Book | ✓ |
| DELETE | /books/{book_id} Delete Book | ✓ |

## Try it: 測試 API

- 找到綠色的「POST」並展開，找到右邊的「Try it out」，輸入參數，再按下下方藍色的「Execute」



# Try it: 測試 API

- 出現「201」表示成功

Curl

```
curl -X 'POST' \  
  'https://render-test-j2u3.onrender.com/books' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "title": "哈利波特",  
    "author": "JK羅琳",  
    "published_year": 9999  
  }'
```

Request URL

<https://render-test-j2u3.onrender.com/books>

Server response

| Code | Details                                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <p>Response body</p> <pre>{<br/>  "title": "哈利波特",<br/>  "author": "JK羅琳",<br/>  "published_year": 9999,<br/>  "id": 1<br/>}</pre> |

# Try it: 測試 API

- 找到藍色的「GET」「/books」並展開，找到右邊的「Try it out」，按下下方藍色的「Execute」

The screenshot shows an API testing tool interface. At the top, there's a tab labeled 'default'. Below it, a list of endpoints is shown. The first endpoint is 'GET / Hello'. The second endpoint is 'GET /books List Books', which is expanded. Under the expanded endpoint, there's a 'Parameters' section with the text 'No parameters'. To the right of the 'Parameters' section is a red 'Cancel' button. Below the 'Parameters' section, there's a blue 'Execute' button and a grey 'Clear' button. The 'Execute' button is highlighted with a red rectangle. Below the buttons, there's a 'Responses' section with the text 'Curl'.



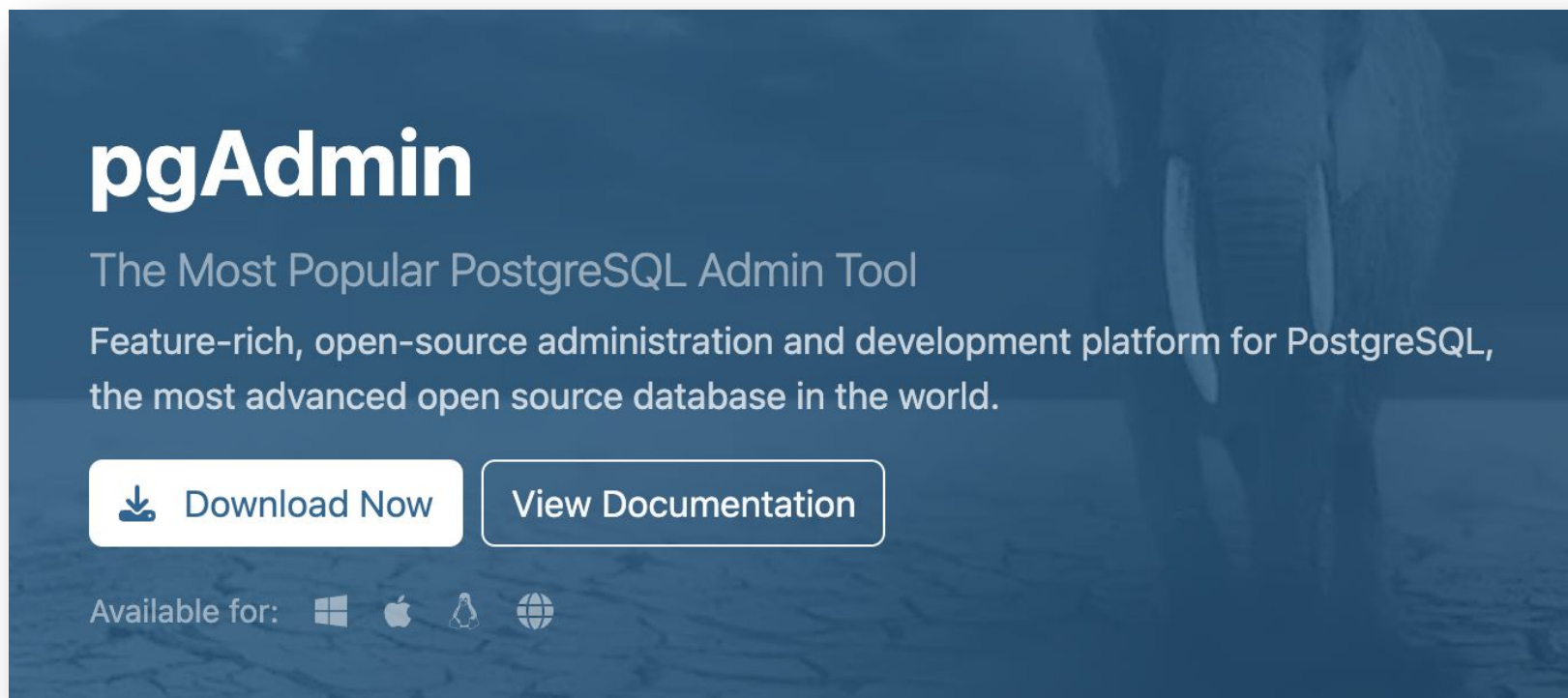
# Try it: 測試 API

- 出現剛剛新增的資料

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div>Response body</div> <pre>[   {     "title": "哈利波特",     "author": "JK羅琳",     "published_year": 9999,     "id": 1   } ]</pre> <div>Response headers</div> <pre>alt-svc: h3=":443"; ma=86400 cf-cache-status: DYNAMIC cf-ray: a1bb5bcfa8942beb-TPE content-encoding: br content-length: 79 content-type: application/json date: Wed, 15 Jul 2026 20:01:22 GMT priority: u=1,i rndr-id: be4abc16-6ee1-4ff5 server: cloudflare server-timing: cfExtPri vary: Accept-Encoding x-render-origin-server: uvicorn</pre> |

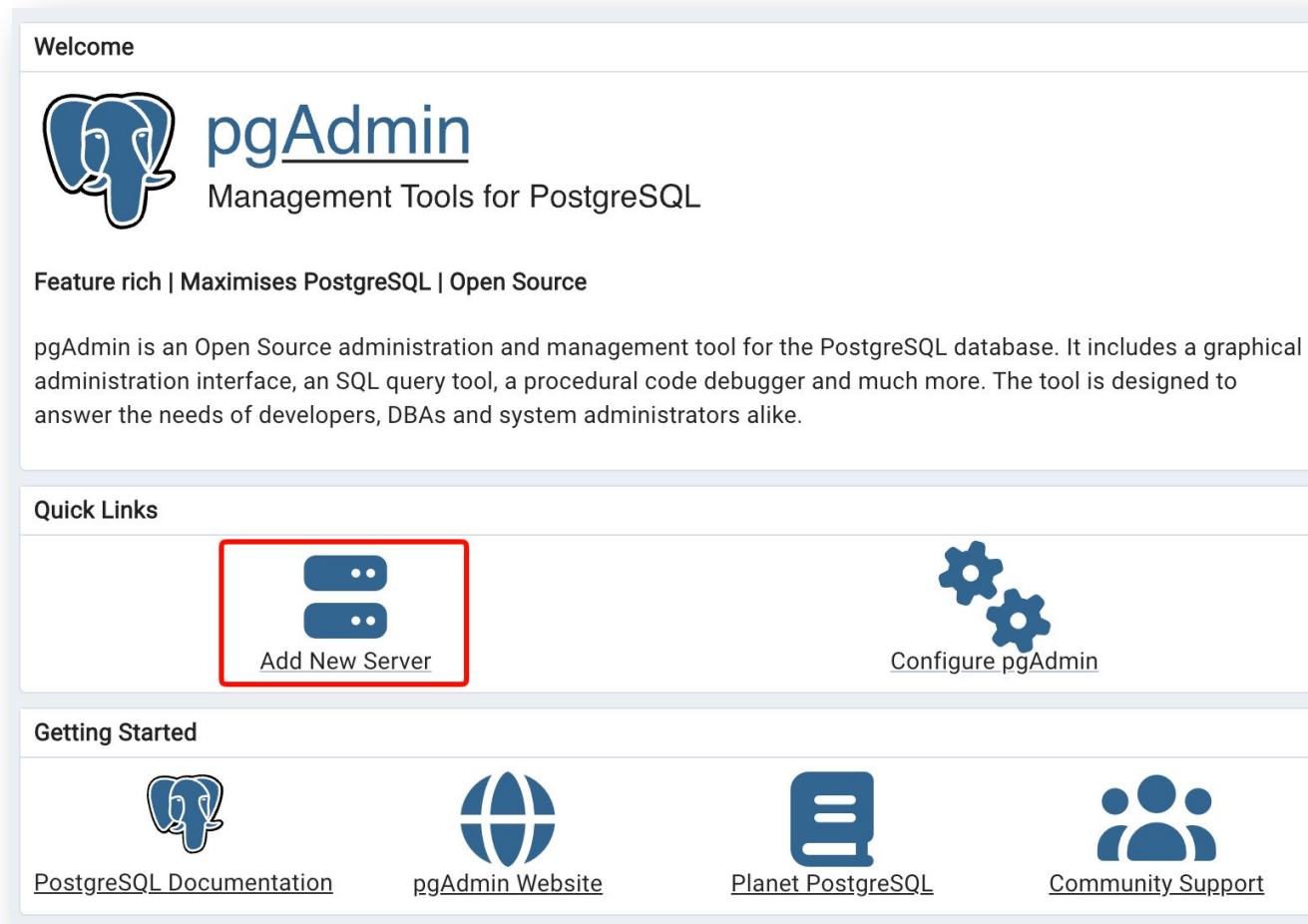
## Try it: 使用 pgadmin 連線資料庫

- pgAdmin 是 PostgreSQL 資料庫最受歡迎的官方開源圖形化管理工具。它提供視覺化介面，讓開發者和管理員免於使用純指令，輕鬆執行建立資料庫、編寫與執行 SQL 查詢、管理權限及監控伺服器效能等日常任務。
- <https://www.pgadmin.org/>



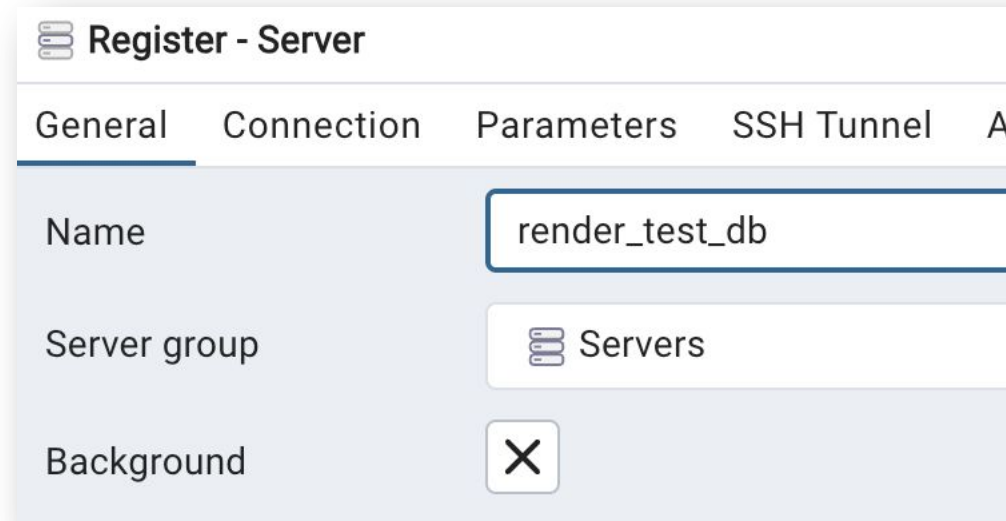
# Try it: 使用 pgadmin 連線資料庫

- pgAdmin 連線方式：
  - 點擊「Add New Server」



# Try it: 使用 pgadmin 連線資料庫

- pgAdmin 連線方式：
  - 輸入連線名稱

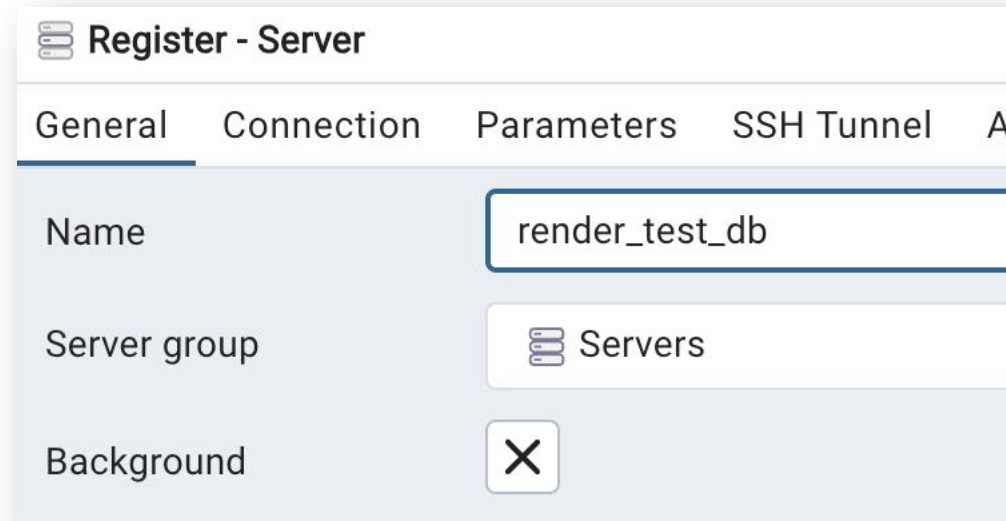


The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'General' tab selected. The dialog has five tabs: General, Connection, Parameters, SSH Tunnel, and Advanced. The 'General' tab contains three fields: 'Name' with the value 'render\_test\_db', 'Server group' with a dropdown menu showing 'Servers', and 'Background' with a checkbox that is currently unchecked.

| Register - Server                                         |                          |
|-----------------------------------------------------------|--------------------------|
| General   Connection   Parameters   SSH Tunnel   Advanced |                          |
| Name                                                      | render_test_db           |
| Server group                                              | Servers                  |
| Background                                                | <input type="checkbox"/> |

# Try it: 使用 pgadmin 連線資料庫

- pgAdmin 連線方式：
  - 輸入連線名稱

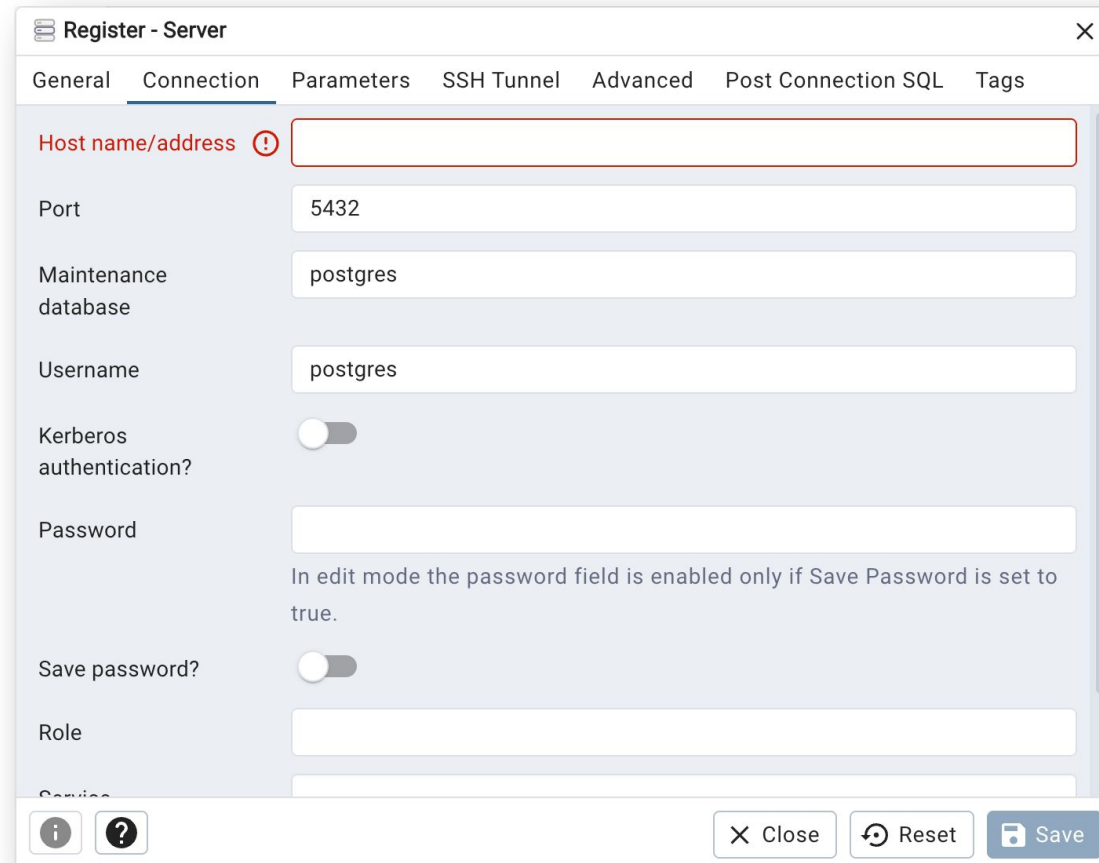


The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'General' tab selected. The dialog has five tabs: General, Connection, Parameters, SSH Tunnel, and Advanced. The 'General' tab contains three fields: 'Name' with the value 'render\_test\_db', 'Server group' with a dropdown menu showing 'Servers', and 'Background' with a checkbox that is currently unchecked.

| Register - Server |                          |
|-------------------|--------------------------|
|                   | General                  |
| Name              | render_test_db           |
| Server group      | Servers                  |
| Background        | <input type="checkbox"/> |

# Try it: 使用 pgadmin 連線資料庫

- pgAdmin 連線方式：
  - 切換到「Connection」



The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'Connection' tab selected. The dialog has several tabs: General, Connection, Parameters, SSH Tunnel, Advanced, Post Connection SQL, and Tags. The 'Connection' tab contains the following fields and controls:

- Host name/address**: A text input field with a red warning icon to its left.
- Port**: A text input field containing the value '5432'.
- Maintenance database**: A text input field containing the value 'postgres'.
- Username**: A text input field containing the value 'postgres'.
- Kerberos authentication?**: A toggle switch that is currently turned off.
- Password**: A text input field. Below it, a note states: 'In edit mode the password field is enabled only if Save Password is set to true.'
- Save password?**: A toggle switch that is currently turned off.
- Role**: A text input field.
- Service**: A text input field.

At the bottom of the dialog, there are three buttons: 'Close', 'Reset', and 'Save'.

# Try it: 使用 pgadmin 連線資料庫

- 回到「render\_test\_db」, 並找到「Connections」

### Connections

|                                                                       |                            |      |
|-----------------------------------------------------------------------|----------------------------|------|
| <b>Hostname</b><br>An internal hostname used by your Render services. | dpg-d9bu0g8js32c73dd844g-a | 📄    |
| <b>Port</b>                                                           | 5432                       | 📄    |
| <b>Database</b>                                                       | render_test_db_x3lv        | 📄    |
| <b>Username</b>                                                       | render_test_db_x3lv_user   | 📄    |
| <b>Password</b>                                                       | .....                      | 👁️ 📄 |
| <b>Internal Database URL</b>                                          | .....                      | 👁️ 📄 |
| <b>External Database URL</b>                                          | .....                      | 👁️ 📄 |
| <b>PSQL Command</b>                                                   | .....                      | 👁️ 📄 |

## Try it: 使用 pgadmin 連線資料庫

- 先複製「**External** Database URL」
- 外部網址的部分，其構成為這樣子  
:postgres://[username]:[password]@[**hostname**].**[伺服器位置].render.com**/[database]
- 我們只需要紅色的部分作為「Host name」



# Try it: 使用 pgadmin 連線資料庫

Register - Server

General

Connection

Parameters

SSH Tunnel

Advanced

Host name/address

dpg-cmdlmped3nmc73dl2du0-a.singapore-postgres.render.com

外部連結的部分字串

Port

5432

連結端口

Maintenance database

quiz\_database\_7f7i

DB名稱

Username

king610160

DB使用者

Kerberos authentication?

☐

Password

.....

連線資料庫密碼

Save password?

☐

Role

Service

i

?

Close

Reset

Save

# Try it: 使用 pgadmin 連線資料庫

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree structure of databases, including 'render\_test\_db' and 'public'. The 'public' schema is expanded, showing various database objects. The main panel shows a query window with the following SQL query:

```
SELECT * FROM public.books
ORDER BY id ASC LIMIT 100
```

The query results are displayed in the 'Data Output' tab, showing a single row of data. The columns are highlighted with a red box:

| id | title | author | published_year |
|----|-------|--------|----------------|
| 1  | 哈利波特  | JK羅琳   | 9999           |

The status bar at the bottom indicates 'Total rows: 1', 'Query complete 00:00:01.003', and 'Ln 1, Col 1'.

# 階段提示 | 功能升級完成, 接著進入系統設計

我們已經知道網站「看起來怎麼運作」。

接下來要理解資料實際經過哪些地方。

你不需要自己寫後端或資料庫程式碼,

**但需要能判斷:**

- 哪一部分應負責什麼？
- 哪些資料要保存？
- 哪個環節最可能失敗？

# 開發前準備

# 1. Agent、Tool、Skill：像一個會做事的專案團隊

三者不是同義詞：一個執行動作、一個固定流程、一個負責判斷。

| 概念    | 生活化理解                 |
|-------|-----------------------|
| Tool  | 像單一工具：查資料、改檔案、執行測試    |
| Skill | 像標準作業流程：規定順序、檢查點與回報方式 |
| Agent | 像專案助理：讀背景、判斷下一步、安排工作  |

人仍負責：目標、限制、批准、驗收與風險決定。

# Tool 做一個動作; Skill 管理一段工作流程

## Tool: 一次完成一個動作

- 例子: 讀檔、查資料、執行測試
- 檢查重點: 這個動作的結果正確嗎?

## Skill: 依照順序完成多個步驟

- 例子: 規劃 → 實作 → 驗收 → 完成回報
- 檢查重點: 批准點、限制與停止條件清楚嗎?

**判斷關鍵: 你要的是「做一下」, 還是「照流程完成」?**

## 實作：建立專案所需文件的 skills

- **建立需求規格書：**「建立一個skill，這個skill是讓AI做為一個需求訪談者，透過問5個最核心的問題來建立需求規格書，並將文件放在/doc資料夾，名為 requirements.md」
- **建立架構設計 說明文件：**「建立一個 skill，這個 skill 會讀取 /doc/requirements.md 的需求規格書，分析系統需求後產生架構設計文件，並將文件放在 /doc 資料夾，名為 architecture.md。文件需包含系統架構、核心模組職責、資料流、資料庫設計建議、API 設計建議、技術選型、安全性、部署方式，以及待確認事項；系統架構圖請使用 Mermaid。若找不到 /doc/requirements.md，請停止執行並提示使用者先完成需求訪談。不得自行補足未確認的需求，應列入「待確認事項」。」

## 實作：建立專案所需文件的 skills

- **建立資料模型文件：**「建立一個 skill, 這個 skill 會讀取 /doc/requirements.md 與 /doc/architecture.md, 依據需求規格與系統架構設計資料模型文件, 並將文件放在 /doc 資料夾, 名為 data-model.md。文件需包含核心實體、各實體欄位與資料型別、主鍵與外鍵、實體關聯、資料驗證規則、索引建議、資料生命週期、敏感資料處理方式, 以及待確認事項; 實體關聯圖請使用 Mermaid ER Diagram。若找不到 /doc/requirements.md 或 /doc/architecture.md, 請停止執行並提示使用者先完成前置文件。不得自行假設未確認的業務規則; 任何推測皆須標示為「待確認事項」。資料模型應使用通用關聯式資料庫概念描述, 並在需要時提供對應的 SQL 資料型別建議。」



## 2. 建立開發規範

**建立開發規範:**「建立開發規範:「建立一個 skill, 此 skill 會讀取 /doc/requirements.md、/doc/architecture.md 與 /doc/data-model.md, 建立供 AI 開發代理與開發者共同遵守的專案規範。

1. 後端使用 FastAPI, 前端使用 React;本地開發資料庫使用 SQLite, 正式部署於 Render.com, 正式資料庫使用 Render PostgreSQL。
2. 規範必須禁止將 API Key、密碼、Token、連線字串及其他機密資訊提交至 Git 或硬編碼於程式中;須使用環境變數與 .env.example 說明所需設定, 並將 .env 加入 .gitignore。
3. 請在專案根目錄建立 AGENTS.md, 作為所有 AI 工具遵循的開發與協作規範;若需要相容 Claude, 可額外建立 CLAUDE.md, 內容應指向或同步 AGENTS.md 的規範。
4. 請建立 /doc/project-memory.md, 用於記錄長期有效的專案決策、技術選型、重要限制與已確認的業務規則;文件必須包含更新規則:僅在確認新決策、修改既有決策或發現重要限制時更新, 且每次更新須記錄日期、原因與影響範圍。
5. 請讀取 /doc/architecture.md, 建立 /doc/todo.md, 列出所有待開發需求、相依關係、驗收條件與優先級, 並依優先級排序。每個項目需包含狀態(todo / In Progress / Done), 且不得把未確認需求列為可直接開發項目。

若前置文件不存在, 請停止並告知缺少哪些文件。產生的規範與待辦事項皆以繁體中文撰寫。」

### 3. 建立除錯 skill

建立 Vibe Coding 除錯引導 skill:「建立一個 skill, 讓 AI 擔任初學者友善的除錯教練, 協助對程式設計、網頁開發與軟體工程觀念尚不熟悉的使用者, 以 Vibe Coding 的方式理解、定位並修正問題。

此 skill 的目標不是只提供可直接貼上的修正程式碼, 而是以容易理解的繁體中文, 循序引導使用者完成除錯, 讓使用者知道問題發生的原因、修正方式與如何自行驗證。

除錯流程必須遵循:

1. 先請使用者提供錯誤訊息、相關程式碼、執行步驟、預期結果與實際結果; 若資訊不足, 一次最多提出 3 個最關鍵的補充問題。
2. 先用白話說明問題可能屬於前端、後端、資料庫、API 串接、環境設定或部署等哪一類。
3. 將錯誤訊息翻譯成初學者能理解的語言, 說明它通常代表什麼。
4. 引導使用者用最小可行的檢查步驟確認原因; 每一步都要說明「要做什麼、為什麼做、預期看到什麼」。
5. 在確認原因後, 提供最小範圍的修正建議與完整可替換程式碼, 清楚標示應修改的檔案與位置。
6. 提供修正後的驗證步驟, 以及若仍失敗時應提供哪些資訊。
7. 結尾以簡短方式整理本次學到的程式或網頁開發觀念。

回覆格式應固定包含:

- 問題白話版
- 最可能的原因
- 先做這些檢查
- 修改方式
- 如何確認已修好
- 本次學到什麼

互動原則:

- 不預設使用者懂終端機、Git、HTTP、API、資料庫、環境變數或框架術語; 首次出現時須用一句白話解釋。
- 避免一次提供太多方案; 優先提出最可能且風險最低的處理方式。

## 4. 建立測試文件的 skill

建立測試文件 skill:「建立一個 skill, 此 skill 會讀取 /doc/requirements.md、/doc/architecture.md、/doc/data-model.md 與 /doc/todo.md, 依據需求、架構、資料模型與開發待辦事項, 產生完整的測試規格文件, 並將文件放在 /doc 資料夾, 名為 test-plan.md。

文件需包含:

- 測試目標、範圍與不在測試範圍內的項目
- 測試策略: 單元測試、整合測試、API 測試、前端元件測試、端對端測試、手動驗收測試
- 測試環境與測試資料規則, 明確區分本機 SQLite 與正式 Render PostgreSQL 環境
- 每項需求對應的測試案例, 包含案例編號、前置條件、測試步驟、輸入資料、預期結果、優先級與驗收標準
- FastAPI 後端的 API 測試重點: 成功、驗證失敗、未授權、找不到資源與伺服器錯誤等情境
- React 前端測試重點: 畫面呈現、使用者互動、表單驗證、載入／錯誤狀態與 API 串接
- 資料庫測試重點: 欄位限制、關聯完整性、資料驗證、遷移與索引
- 安全性測試: 機密資訊不得出現在程式碼、Git 或前端畫面; 驗證登入、權限與輸入資料處理
- 缺陷回報格式與測試完成標準
- 待確認事項

請以 Markdown 表格呈現主要測試案例。測試案例必須能追溯到需求或待辦事項, 並依需求優先級排序。

若缺少任何前置文件, 請停止執行並指出缺少的文件。不得自行假設未確認的需求或行為; 不確定之處須列入「待確認事項」文件以繁體中文撰寫。

# 開發外送系統

# Try it: 建立一個外送系統

## 1. 用需求訪談 skill 釐清外送系統需求

先下這樣的任務：

請使用需求訪談 skill，針對「餐點外送系統」進行需求訪談。系統包含消費者、餐廳與管理員角色；消費者可瀏覽餐廳與餐點、加入購物車、建立訂單與查看訂單狀態。請透過 5 個最核心問題逐步訪談，完成後產生 `/doc/requirements.md`。

## 2. 用架構設計 skill 產生架構文件

請使用架構設計 skill，讀取 `/doc/requirements.md`，產生外送系統的 `/doc/architecture.md`。系統採用 FastAPI、React、SQLite 本地開發、Render PostgreSQL 正式環境。請特別設計使用者、餐廳、菜單、購物車、訂單與訂單狀態模組。

## 3. 用資料模型 skill 產生資料庫設計

請使用資料模型 skill，讀取 `/doc/requirements.md` 與 `/doc/architecture.md`，產生 `/doc/data-model.md`。請涵蓋使用者、餐廳、餐點分類、餐點、購物車、購物車項目、訂單、訂單項目與訂單狀態紀錄，並以 Mermaid ER Diagram 呈現關聯。

# Try it: 建立一個外送系統

## 4. 用開發規範 skill 建立共同工作規則與待辦清單

請使用開發規範 skill，讀取 `/doc/requirements.md`、`/doc/architecture.md` 與 `/doc/data-model.md`，建立 `AGENTS.md`、`/doc/project-memory.md` 與 `/doc/todo.md`。待辦請依優先級安排，優先完成可建立訂單的最小可行產品（MVP）。

外送系統的 MVP 順序通常會是：

1. 使用者註冊 / 登入
2. 餐廳與餐點瀏覽
3. 購物車
4. 建立訂單
5. 查詢訂單狀態
6. 餐廳端更新訂單狀態
7. 管理員管理餐廳與餐點



# Try it: 建立一個外送系統

## 5. 依照 `todo.md` 逐項開發

每次只做一個明確功能，例如：

請依照 `/doc/todo.md` 與 `AGENTS.md`，實作優先級最高且尚未完成的功能。先閱讀相關文件與既有程式碼，再提出簡短實作計畫；完成後更新 `/doc/todo.md` 的狀態與 `/doc/project-memory.md` 的重要決策。

## 6. 用測試文件 skill 建立驗收依據

在 MVP 開發前或完成後都可以執行：

請使用測試文件 skill，讀取 `/doc/requirements.md`、`/doc/architecture.md`、`/doc/data-model.md` 與 `/doc/todo.md`，產生 `/doc/test-plan.md`。請優先涵蓋登入、餐點瀏覽、購物車、建立訂單、訂單金額計算與訂單狀態更新。

## 7. 遇到錯誤時用 Vibe Coding 除錯 skill

請使用 Vibe Coding 除錯引導 skill 協助我處理問題。我是初學者，請先用白話解釋錯誤，再一步一步引導我檢查。以下是錯誤訊息、相關程式碼、預期結果與實際結果：.....

# 個人每日回饋 | 整理今天學到與做到的事

請填寫表單，讓我知道你今天學習的成果！

<https://forms.gle/Gz5PaRPLVQTjtPts9>

